

1. Generación de Polinomios

Como parámetros se reciben una instancia de un campo de Galois y una lista o tupla de coeficientes pertenecientes a dicho campo, ordenados en forma decreciente según su grado.

1.1. Verificaciones

Se realizan cuatro verificaciones. En primer lugar, se comprueba que el campo recibido sea efectivamente una instancia de un campo de Galois. Luego, se verifica que los coeficientes estén contenidos en una lista o tupla y que esta no se encuentre vacía. Finalmente, se comprueba que todos los coeficientes sean elementos de un mismo campo de Galois y que dicho campo coincida con el asociado a la instancia.

Para determinar si dos polinomios pertenecen al mismo campo de Galois, se verifica que la variable `field` de ambos corresponda a campos del mismo orden y generados a partir del mismo polinomio primitivo.

1.2. Ceros a la izquierda

Se permite recibir una lista de coeficientes con ceros a la izquierda, los cuales se eliminan automáticamente. Dado que los coeficientes se encuentran ordenados en forma decreciente según su grado, se descartan sucesivamente hasta encontrar el primer coeficiente no nulo.

1.3. Constructor

El constructor realiza las verificaciones correspondientes, almacena los parámetros recibidos en variables locales y elimina los ceros a la izquierda de la lista de coeficientes.

```
1 from galois_field import GaloisField
2
3 class GFPoly:
4
5     def __init__(self, field, coefficients):
6         if not isinstance(field, GaloisField):
7             raise TypeError(
8                 "field debe ser una instancia de GaloisField."
9             )
10
11         if not isinstance(coefficients, (list, tuple)):
12             raise TypeError(
13                 "Los coeficientes deben entregarse en una lista o tupla."
14             )
15
16         if len(coefficients) == 0:
17             raise ValueError(
18                 "La lista de coeficientes no puede estar vacía."
19             )
20
21         self.field = field
22
23         for coefficient in coefficients:
24             self._check_coefficient(coefficient)
25
26         self.coefficients = list(coefficients)
27
28         self._remove_leading_zeros()
29
30     def _check_coefficient(self, coefficient):
31
```

```

32     if not isinstance(coefficient, GaloisField):
33         raise TypeError(
34             "Todos los coeficientes deben ser elementos "
35             "de GaloisField."
36         )
37
38     if (
39         coefficient.m != self.field.m
40         or coefficient.primitive_polynomial
41         != self.field.primitive_polynomial
42     ):
43         raise ValueError(
44             "Todos los coeficientes deben pertenecer "
45             "al mismo campo."
46         )
47
48     def _check_polynomial(self, other):
49
50         if not isinstance(other, GFPoly):
51             raise TypeError(
52                 "La operaci n requiere otro polinomio GFPoly."
53             )
54
55         if (
56             self.field.m != other.field.m
57             or self.field.primitive_polynomial
58             != other.field.primitive_polynomial
59         ):
60             raise ValueError(
61                 "Los polinomios pertenecen a campos diferentes."
62             )
63
64     def _remove_leading_zeros(self):
65
66         while (
67             len(self.coefficients) > 1
68             and self.coefficients[0].value == 0
69         ):
70             self.coefficients.pop(0)

```

2. Operaciones

2.1. Funciones Auxiliares útiles

A la hora de realizar operaciones entre polinomios de un mismo campo de Galois resulta útil conocer el grado de los polinomios y particularmente si alguno de los polinomios involucrados es el polinomio nulo. Como ya se eliminaron los coeficientes nulos a la izquierda, el grado del polinomio sera el largo de la lista de coeficientes menos uno.

```

1 def degree(self):
2
3     if self.is_zero():
4         return -1
5
6     return len(self.coefficients) - 1
7
8 def is_zero(self):
9
10    return (
11        len(self.coefficients) == 1

```

```

12         and self.coefficients[0].value == 0
13     )

```

2.2. Suma y Resta

En los campos de Galois $GF(2^m)$, la suma y la resta son operaciones equivalentes. Para realizarlas, primero se determina el grado máximo entre ambos polinomios. Luego, se agregan ceros a la izquierda de la lista de coeficientes del polinomio de menor grado, de modo que los coeficientes de ambos queden alineados según las potencias correspondientes.

Finalmente, se recorren ambas listas coeficiente a coeficiente y se realiza la suma entre los elementos del campo de Galois ubicados en cada posición.

```

1 def __add__(self, other):
2
3     self._check_polynomial(other)
4
5     length = max(
6         len(self.coefficients),
7         len(other.coefficients)
8     )
9
10    zero = self.field[0]
11
12    coefficients_a = (
13        [zero] * (length - len(self.coefficients))
14        + self.coefficients
15    )
16
17    coefficients_b = (
18        [zero] * (length - len(other.coefficients))
19        + other.coefficients
20    )
21
22    result = [
23        a + b
24        for a, b in zip(coefficients_a, coefficients_b)
25    ]
26
27    return GFPoly(self.field, result)
28
29 def __sub__(self, other):
30
31    return self + other

```

2.3. Multiplicación

Si alguno de los polinomios es nulo, el resultado de la multiplicación es también el polinomio nulo.

En caso contrario, la cantidad de coeficientes del polinomio resultante se determina como la suma de las cantidades de coeficientes de ambos polinomios, menos uno.

Luego, se realiza una doble iteración sobre los coeficientes de ambos polinomios. Para cada par de coeficientes, se calcula su producto en el campo de Galois y se suma el resultado al coeficiente correspondiente del polinomio resultante, de acuerdo con el grado asociado a dicha multiplicación.

```

1 def __mul__(self, other):
2
3     self._check_polynomial(other)
4

```

```

5     if self.is_zero() or other.is_zero():
6         return GFPolynomial(self.field, [self.field[0]])
7
8     result_length = (
9         len(self.coefficients)
10        + len(other.coefficients)
11        - 1
12    )
13
14    result = [
15        self.field[0]
16        for _ in range(result_length)
17    ]
18
19    for i, coefficient_a in enumerate(self.coefficients):
20        for j, coefficient_b in enumerate(other.coefficients):
21            result[i + j] = (
22                result[i + j]
23                + coefficient_a * coefficient_b
24            )
25
26    return GFPolynomial(self.field, result)

```

2.4. División

En primer lugar, se verifica que el divisor no sea el polinomio nulo. Además, si el grado del dividendo es menor que el grado del divisor, el cociente es el polinomio nulo y el resto coincide con el dividendo.

En caso contrario, se aplica el algoritmo de división de polinomios. La cantidad de coeficientes del cociente puede determinarse a partir de la diferencia entre los grados del dividendo y del divisor. Inicialmente, se asigna el dividendo al resto y se establece el cociente como el polinomio nulo.

Luego, se ejecuta un loop mientras el resto sea distinto del polinomio nulo y su grado sea mayor o igual que el grado del divisor. En cada iteración, se calcula el coeficiente principal como el cociente entre el coeficiente de mayor grado del resto y el coeficiente de mayor grado del divisor. La diferencia entre ambos grados determina la posición correspondiente de dicho coeficiente en el cociente.

A partir de este coeficiente y de la diferencia de grados, se construye un polinomio auxiliar. Este se suma al cociente y, luego de multiplicarlo por el divisor, se suma al resto. Dado que en $GF(2^m)$ la suma y la resta son equivalentes, esta operación elimina el término de mayor grado del resto.

Una vez finalizado el loop, se obtienen el cociente y el resto de la división.

```

1 def divmod(self, divisor):
2
3     self._check_polynomial(divisor)
4
5     if divisor.is_zero():
6         raise ZeroDivisionError(
7             "No se puede dividir por el polinomio nulo."
8         )
9
10    if self.degree < divisor.degree:
11        quotient = GFPolynomial(
12            self.field,
13            [self.field[0]]
14        )
15
16        remainder = GFPolynomial(
17            self.field,
18            self.coefficients
19        )

```

```

20
21     return quotient, remainder
22
23     quotient_length = self.degree - divisor.degree + 1
24
25     quotient_coefficients = [
26         self.field[0]
27         for _ in range(quotient_length)
28     ]
29
30     remainder = GFPoly(
31         self.field,
32         self.coefficients
33     )
34
35     while (
36         not remainder.is_zero()
37         and remainder.degree >= divisor.degree
38     ):
39         leading_coefficient = (
40             remainder.coefficients[0]
41             / divisor.coefficients[0]
42         )
43
44         degree_difference = (
45             remainder.degree - divisor.degree
46         )
47
48         quotient_position = (
49             quotient_length - degree_difference - 1
50         )
51
52         quotient_coefficients[quotient_position] = (
53             leading_coefficient
54         )
55
56         term_coefficients = (
57             [leading_coefficient]
58             + [self.field[0]] * degree_difference
59         )
60
61         term = GFPoly(
62             self.field,
63             term_coefficients
64         )
65
66         remainder = remainder + term * divisor
67
68     quotient = GFPoly(
69         self.field,
70         quotient_coefficients
71     )
72
73     return quotient, remainder
74
75 def __floordiv__(self, other):
76
77     quotient, _ = self.divmod(other)
78
79     return quotient
80
81 def __mod__(self, other):
82

```

```

83     _, remainder = self.divmod(other)
84
85     return remainder

```

2.5. Escalado y Evaluación

El escalado consiste en multiplicar todos los coeficientes del polinomio por un mismo elemento del campo de Galois.

Para evaluar un polinomio

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0,$$

se utiliza el método de Horner, que permite reescribirlo de forma anidada como

$$P(x) = (\cdots ((a_n x + a_{n-1}) x + a_{n-2}) x + \cdots + a_1) x + a_0.$$

De esta manera, la evaluación se realiza recorriendo los coeficientes en orden decreciente de grado y actualizando sucesivamente

$$r \leftarrow r \cdot x + a_i.$$

El método evita calcular explícitamente las potencias de x , por lo que requiere únicamente n multiplicaciones y n sumas en el campo $GF(2^m)$.

Además, se agregó que se pueda evaluar el polinomio directamente usando $(,)$.

```

1 def scale(self, scalar):
2
3     self._check_coefficient(scalar)
4
5     result = [
6         coefficient * scalar
7         for coefficient in self.coefficients
8     ]
9
10    return GFPolynomial(self.field, result)
11
12 def evaluate(self, x):
13
14     self._check_coefficient(x)
15
16     result = self.field[0]
17
18     for coefficient in self.coefficients:
19         result = result * x + coefficient
20
21     return result
22
23 def __call__(self, x):
24
25     return self.evaluate(x)

```

2.6. Generación de un Polinomio a partir de sus Raíces

Para generar un polinomio a partir de sus raíces, se comienza con el polinomio constante 1. Luego, para cada raíz, se verifica que esta pertenezca al campo de Galois correspondiente y se construye el polinomio $X + \text{root}$. Este factor se multiplica por el polinomio acumulado, inicialmente definido como 1.

Al repetir este procedimiento para todas las raíces recibidas, se obtiene el polinomio resultante.

```

1 @classmethod
2 def from_roots(cls, field, roots):
3
4     if not isinstance(roots, (list, tuple)):
5         raise TypeError(
6             "Las ra ces deben entregarse en una lista o tupla."
7         )
8
9     polynomial = cls(
10         field,
11         [field[field.one_value]]
12     )
13
14     one = field[field.one_value]
15
16     for root in roots:
17         if not isinstance(root, GaloisField):
18             raise TypeError(
19                 "Todas las ra ces deben ser elementos "
20                 "de GaloisField."
21             )
22
23         if (
24             root.m != field.m
25             or root.primitive_polynomial
26             != field.primitive_polynomial
27         ):
28             raise ValueError(
29                 "Todas las ra ces deben pertenecer "
30                 "al mismo campo."
31             )
32
33         factor = cls(
34             field,
35             [one, root]
36         )
37
38         polynomial = polynomial * factor
39
40     return polynomial

```

2.7. Comparación y Representación

Para comparar dos polinomios, se verifica que ambos estén definidos sobre el mismo campo de Galois y que sus respectivas listas de coeficientes sean iguales.

Para su representación mediante `print()`, los coeficientes se muestran como m -uplas binarias.

```

1 def __eq__(self, other):
2
3     if not isinstance(other, GFPoly):
4         return False
5
6     same_field = (
7         self.field.m == other.field.m
8         and self.field.primitive_polynomial
9         == other.field.primitive_polynomial
10    )
11
12    return (
13        same_field
14        and self.coefficients == other.coefficients

```

```
15     )
16
17 def __repr__(self):
18
19     coefficients = [
20         format(coefficient.value, f"0{self.field.m}b")
21         for coefficient in self.coefficients
22     ]
23
24     return f"GFPoly({coefficients})"
```